AgentOS Gap Closure Live UI Validation + MAD Analytical Review

Date: 2026-06-16
Repo: `C:\Users\gaged\Documents\AgenOS`

1) Scope and Objective

This review closes the previously identified gap: **no live UI walkthrough had been performed**.
It adds runtime evidence from a live browser pass, then synthesizes a deep proposal set ("MAD" = multi-axis diagnostic) with implementation roadmaps per proposal.

2) Evidence Collected During Gap Closure

2.1 Stack health and endpoint checks

- `pnpm stack:status` reported API/Gateway healthy and command center active on port override 3010.
- API health was reachable and returned `ok:true`.
- Gateway health was reachable and returned `ok:true`.
- Command-center ports 3000/3002/3003/3010 all returned HTTP 500 for direct page load under the running background stack.

2.2 Fresh command-center validation (control sample)

A fresh Next dev server on port `3025` was started from `apps/command-center`, and rendered successfully.

Observed behavior:
- `/` rendered full Forge dashboard and mission controls.
- `/control-gate` rendered with live data and run history.
- `/missions`, `/settings` intermittently showed `Offline / Initializing forge shell` state.
- API proxy itself worked from the same app instance (`/agentos-api/health` returned healthy JSON).

2.3 Runtime smoke confirmation

- `pnpm mission:smoke` passed.
- `pnpm tool:smoke` passed.
- `pnpm test:e2e` failed due to `EADDRINUSE` (port 3010 already in use), not test assertion failure.

3) Gap-Closure Finding (Root Conclusion)

The UI gap is now closed with live evidence:

1. The product **does render and function live** in a fresh command-center instance.
2. The background stack command-center binding has a **runtime fault condition** causing route-level 500s / offline shell behavior.
3. The issue is not a total API outage (API and gateway are healthy), but a **frontend runtime/session-state instability under the long-running stack process topology**.

4) Technical Analysis (MAD: Multi-Axis Diagnostic)

Axis A Connectivity and fallback model

The frontend is intentionally designed for WS-first with polling fallback and eventual offline state:

```13:23:apps/command-center/src/lib/use-agentos-events.ts
function eventsUrl() {
  const base = resolveApiBase();
  if (base.startsWith("http")) {
    return base.replace(/^http/, "ws") + "/events";
  }
  if (typeof window !== "undefined") {
    const proto = window.location.protocol === "https:" ? "wss:" : "ws:";
    return `${proto}//${window.location.host}/agentos-api/events`;
  }
  return "ws://127.0.0.1:8787/events";
}
```

```

```52:68:apps/command-center/src/lib/use-agentos-events.ts
const startPolling = () => {
  setMode("polling");
  ...
  const res = await fetch(`${resolveApiBase()}/dashboard`, { ... });
  if (!res.ok) {
    setMode("offline");
    return;
  }
  ...
};
```

This behavior explains why sections can collapse to offline-shell despite partial backend health.

Axis B — API base/proxy behavior

The command center always uses `/agentos-api` in browser mode:

```1:12:apps/command-center/src/lib/agentos-api.ts
const PUBLIC_API_BASE = process.env.NEXT_PUBLIC_AGENTOS_API_URL ?? "http://localhost:8787";

function browserApiBase() {
  return "/agentos-api";
}

export function resolveApiBase() {
  if (typeof window !== "undefined") {
    return browserApiBase();
  }
  return PUBLIC_API_BASE;
}
```

And Next rewrites map that to API target:

```53:65:apps/command-center/next.config.mjs
const localApiTarget = `http://127.0.0.1:${process.env.AGENTOS_API_PORT ?? 8787}`;
...
const apiProxyTarget =
  process.env.AGENTOS_API_PROXY_TARGET?.trim() ||
  (process.env.NODE_ENV !== "production" ? localApiTarget : null) ||
  process.env.AGENTOS_API_BASE_URL?.trim() ||
  process.env.NEXT_PUBLIC_AGENTOS_API_URL?.trim() ||
  localApiTarget;
```

```91:103:apps/command-center/next.config.mjs
async rewrites() {
  return [
    {
      source: "/agentos-api/:path*",
      destination: `${apiProxyTarget.replace(/\/$/, "")}/:path*`
    }
  ];
}
```

Proxy health worked on 3025 (`/agentos-api/health` good), so instability is likely around route/render lifecycle or stale runtime state rather than rewrite misconfiguration alone.

Axis C — Offline shell behavior in app component

When API status flips false, app intentionally renders offline shell + boot loader:

```735:744:apps/command-center/src/components/local/AgentOSLocalApp.tsx
```
```

```
if (apiOnline === false) {
  return (
    <ForgeDashboardShell ...>
      <div className="alert error-alert">
        AgentOS API is offline...
      </div>
      <ForgeBootLoader compact inline />
    </ForgeDashboardShell>
  );
}
```

This matches the observed "Initializing forge shell / Offline" state on affected routes.

Axis D — Process and port topology

The stack used an override port (`3010`) and had additional listeners on 3000/3002/3003.
Playwright e2e failed with `EADDRINUSE: 3010`, proving active contention in the session topology.

5) Updated Findings (Compared to Prior Audit)

5.1 Confirmed from live validation

- Previous caveat is resolved: **runtime UI behavior is now observed directly**.
- Core surfaces can render correctly in a clean app process.
- Control Gate and run-history interaction are visibly functional.

5.2 New finding introduced by gap closure

- There is an additional operational reliability issue: **background command-center process can enter a 500/offline-render state while API remains healthy**.
- This issue affects confidence in long-running stack stability and explains user-facing inconsistency.

6) MAD Proposals and Technical Roadmaps

Proposal P1 — Route-Resilience Hardening (Frontend Runtime Reliability)

Goal: Eliminate route-level offline-shell false negatives when API is actually reachable.

Plan

1. Add route-level health instrumentation inside command-center (status badge with last successful `/dashboard` poll timestamp + WS state transitions).
2. Separate "API unreachable" from "dashboard payload invalid" and "render exception" states.
3. Add guarded fallback for section pages (`/missions`, `/settings`) to continue rendering last known payload while surfacing a non-blocking warning.
4. Add structured error boundary around `AgentOSLocalApp` section rendering.

Roadmap

- Week 1: Instrumentation + status model
- Week 2: Error-state split + fallback rendering
- Week 3: Route tests for degraded mode

Success criteria

- No full-shell offline collapse if `/health` is reachable and last snapshot is valid.
- Route navigation continues with stale-while-revalidate behavior under temporary WS/poll failures.

Proposal P2 — Stack Port Determinism and Process Hygiene

Goal: Remove multi-port drift and EADDRINUSE friction.

Plan

1. Introduce a preflight check in stack launcher that enforces single authoritative command-center port and kills stale listeners from previous runs.
2. Persist an explicit process manifest with PID ownership and startup timestamps.
3. Add `stack:doctor` command to detect and repair port drift (3000/3002/3003/3010 fanout).
4. Make e2e webServer port configurable from env to avoid collisions.

Roadmap

- Week 1: PID/port preflight
- Week 2: `stack:doctor` + repair integration
- Week 3: CI/e2e dynamic port support

Success criteria

- One active command-center listener per session.
- Zero `EADDRINUSE` failures in local e2e runs.

Proposal P3 – Control-Gate Truthfulness and Session Attribution

Goal: Ensure approval UI state remains trustworthy under transient API conditions.

Plan

1. Persist last known approvals/runs snapshot in session storage with timestamp.
2. Render stale indicator (`last updated x sec ago`) rather than hard offline reset.
3. Strengthen operator attribution flow in UI and API warnings when in local operator fallback.
4. Add synthetic monitor for `/control-gate` route load + data consistency checks.

Roadmap

- Week 1: stale snapshot cache
- Week 2: attribution UX and warnings
- Week 3: synthetic monitor + alert

Success criteria

- Control Gate never appears empty due to transient fetch failures.
- Operator attribution status is always explicit and auditable.

Proposal P4 – Production-Readiness Gate: "No-Fake-Live" + Runtime Stability

Goal: Tie release readiness to both data truth and runtime resiliency.

Plan

1. Add release gate requiring:
   - no synthetic widgets labeled LIVE,
   - stack stability smoke (`/`, `/missions`, `/control-gate`, `/settings`, `/blackbox`),
   - command-center 500-rate = 0 during 15-minute soak.
2. Add a CI job that boots command-center fresh and verifies all above routes return functional snapshots.
3. Publish a `runtime-integrity.md` checklist in docs.

Roadmap

- Week 1: gate definition + checklist
- Week 2: CI route smoke
- Week 3: soak-test automation

Success criteria

- Release blocked if route stability or truthfulness checks fail.
- Dashboard integrity no longer depends on manual observation.

---

7) Priority Stack (Implementation Order)

1. **P2 Port/process determinism** (stabilize execution substrate)
2. **P1 Route-resilience hardening** (prevent false offline rendering)
3. **P3 Control-gate truthfulness** (operator trust)
4. **P4 Release integrity gate** (prevent regression)

8) Immediate Actions (Next 24 Hours)

1. Run `pnpm stack:repair` and capture command-center logs for 500 root-cause trace.
2. Add temporary telemetry logging around `apiOnline` flips and `eventsMode` transitions.
3. Execute repeated route sweep (`/`, `/missions`, `/control-gate`, `/settings`, `/blackbox`) under one stable port.
4. Re-run `pnpm test:e2e` with non-conflicting webServer port.

9) Final Gap-Closure Statement

The "One gap to close" is now completed with runtime evidence: the UI has been validated live, and the result is nuanced but actionable **AgentOS UI is functionally real on clean startup, but long-running stack process/port instability can force false offline states and 500 responses on core routes.**

This does not overturn the prior forensic classification; it sharpens it with a concrete reliability delta and a roadmap to close it.